Execution Manager

Mohamed Saied

Exec in architecture

ara::exec

parent process in Adaptive Autosar
depend on machine state
Startup, running, ...

Machine state differ from soc to other
em fork the app that want to start

EM is responsible for:

Platform Lifecycle Management

Application Lifecycle Management

EM is not responsible for the run-time scheduling

Applications have to report to EM when the initialisation was completed by switching „running“ state

EM terminates applications using SIGTERM
and applications have to report „terminating“ state

Em: responsible on start and shutdown not scheduling

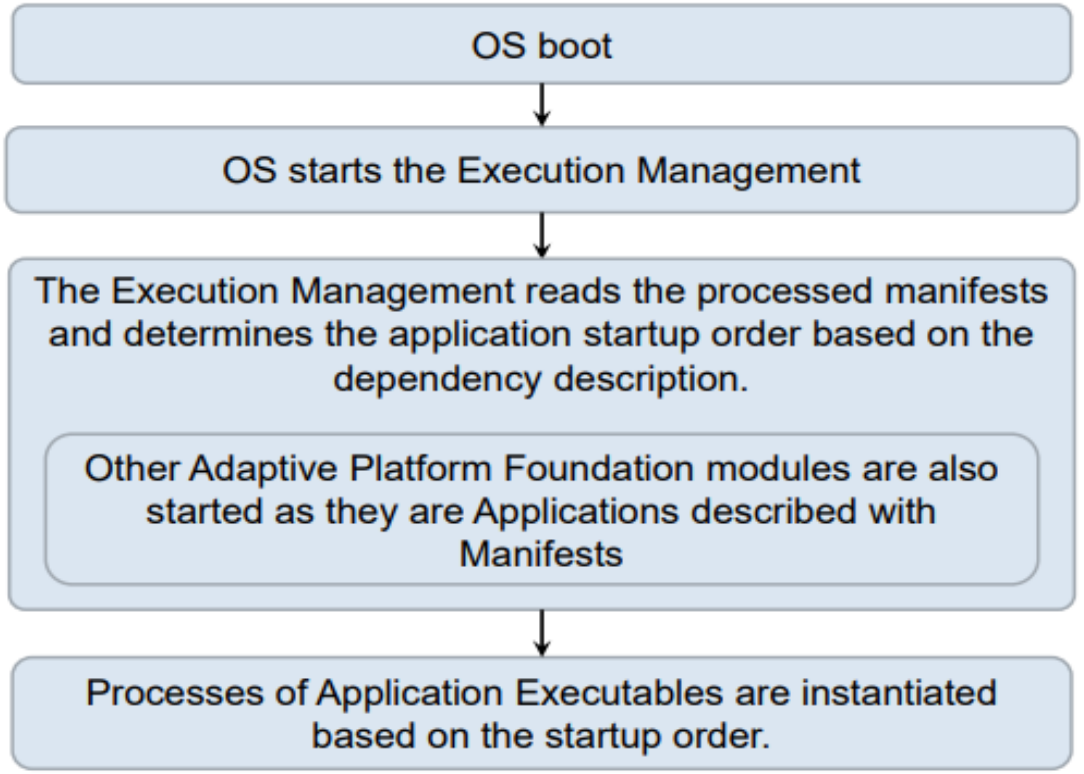


Figure 7.5: Startup sequence

Future and promise
Concurrency -> Asynch in c++

https://www.dexterindustries.com/howto/run-a-program-on-your-raspberry-pi-at-startup/

Startup/Shutdown sequence

Startup and shutdown sequence
- Machine states
- Different according to on chip hardware
  - PowerPC, Intel, ..

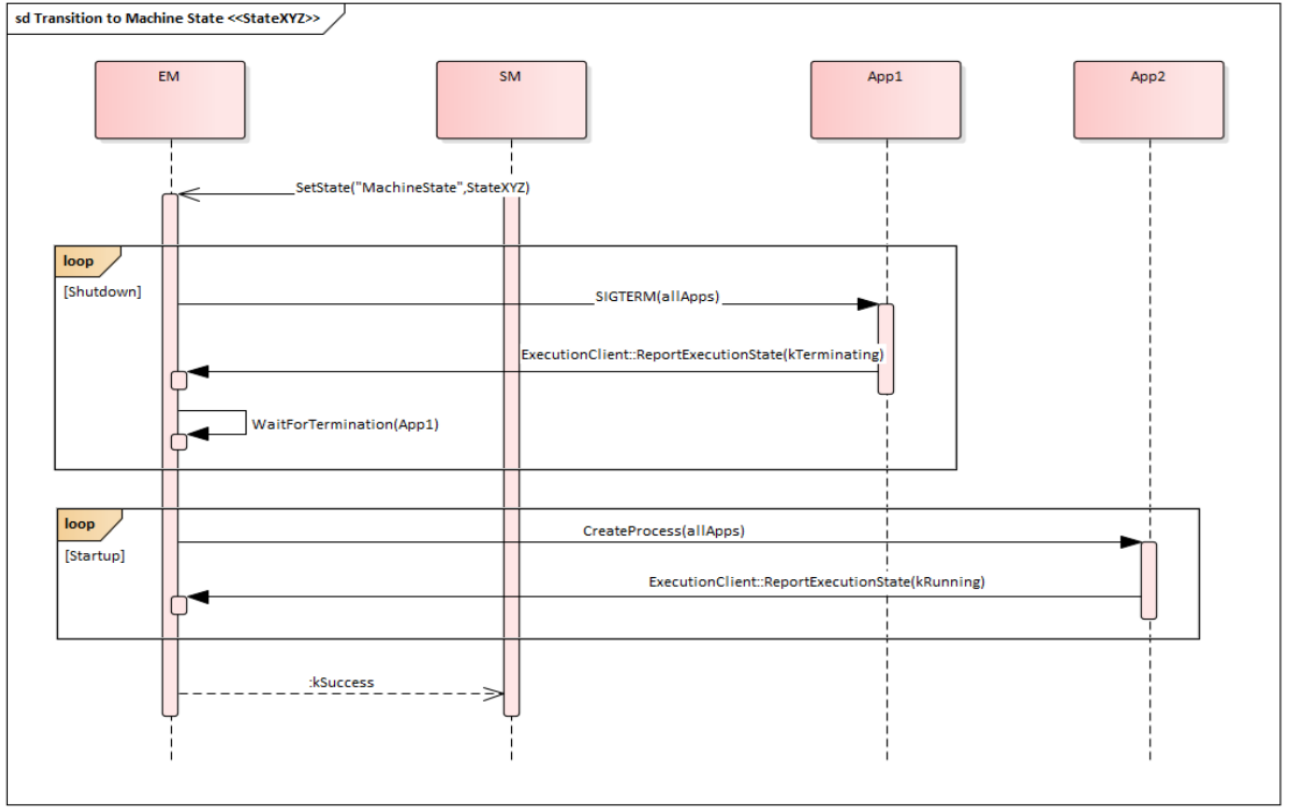


Figure 7.2: State Change Sequence – Transition to machine state `StateXYZ`

concurrency libraries
Asynch lib in cpp

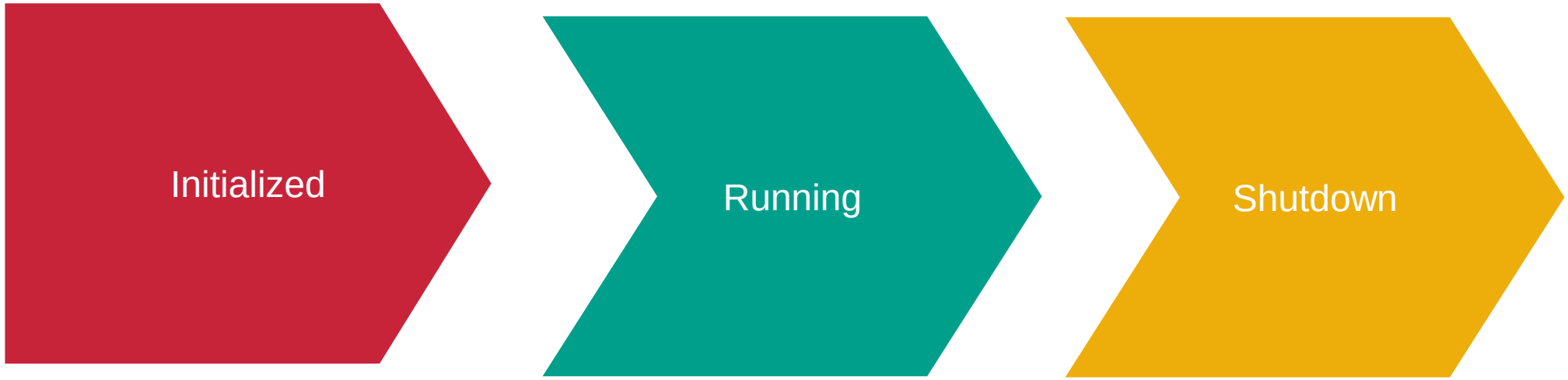
state manager(SM) report the machine state

for example
if we have
10 App

may be 8 run in running state machine
and 2 running in startup or boot machine state

App States

App states differs to machine state



Machine start up sequence

EM -> EM initiates the Machine State : off state > start state (#1) -> EM reports Machine State Startup transition confirmation to State Management (#2)

- (#1) During the transition, Execution Manage- ment requests startup of processes that exist in the Startup Machine State.
- (#2) At that point, EM hands over responsibility for Function Group state management (i.e. initiation of state change requests) to State Management.
- EM is not necessarily the first process launched, Other processes needed by the system may exist,
- Please note that an Application consists of one or more Executables. There- fore to launch an Application, Execution Management starts processes as instances of each Executable.

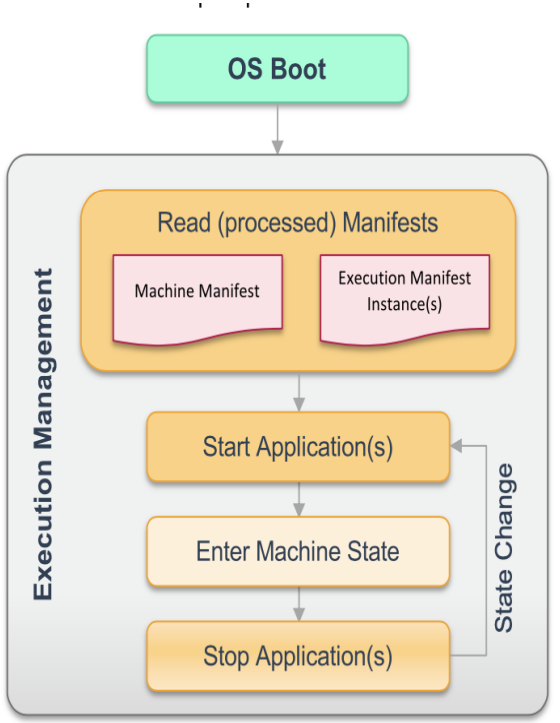


Figure 7.6: Startup sequence

Sigterm may be implemented in OSAL(os abstraction layer)

Platform boundary

- How to end nm?
- Execution Management is started as part of the AUTOSAR Adaptive Platform startup phase and is responsible for starting and terminating processes. -> so we can know that em will terminate nm

Os run init process that run execution manager that starts the applications as child processes

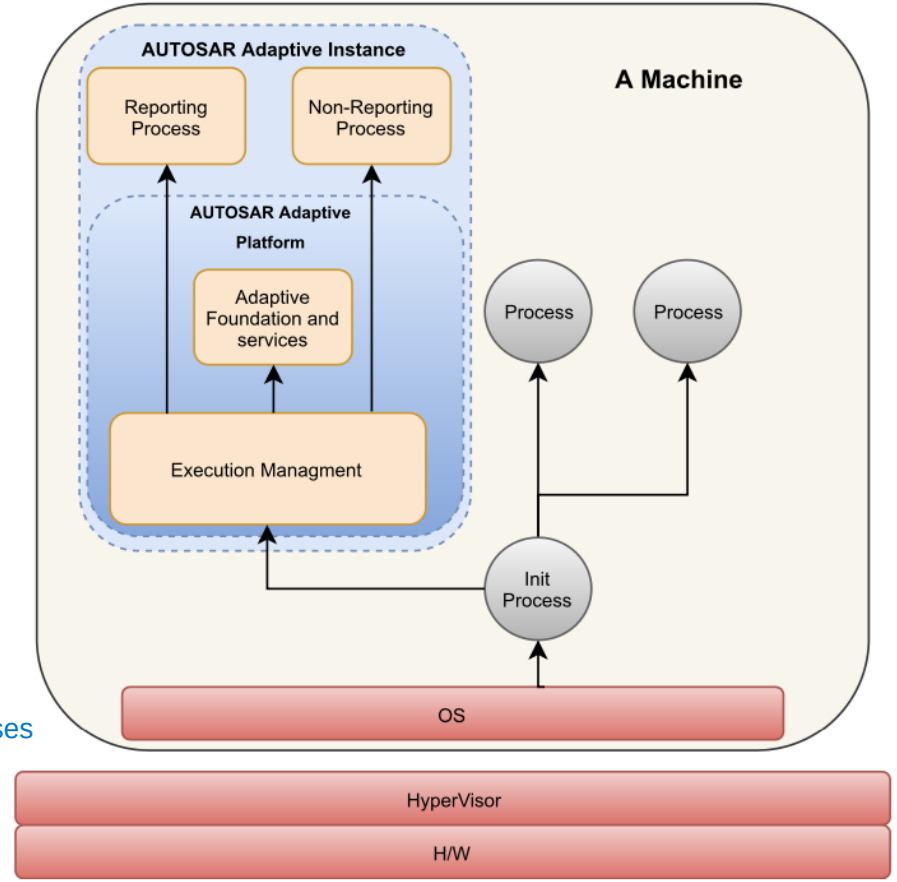


Figure 7.7: AUTOSAR Adaptive Platform Boundary

Termination

- Execution Management does not perform standardized termination handling - the response to receipt of a signal, e.g. SIGTERM, by Execution Management is there- fore implementation defined.

```
int main(){

return 0; // return to OS means that exits success
}
```

Each procees will terminate with some thing

[SWS_EM_01055] Initiation of process termination ⌈Execution Management shall initiate process termination by sending the SIGTERM signal to the process.⌋ *(RS_EM_00103)*

Termination

- Note that from the perspective of Execution Management, requirement [SWS_EM_01055] only requests the initiation of the steps necessary for grace- ful termination under the control of the process.
- Execution ManagementmaysendSIGTERMatanytime,evenbeforetheprocess has reported kRunning state and thus the process is still in the Initializing Process State. On receipt of SIGTERM, a process simply commences the actual termination.-> we can terminate it any time
- Terminating state target :
- save persistent data
- free all internally used resources
- termination with exit status 0 (EXIT_SUCCESS)

process sequenece

Persistent or no volatile memory like diagnosis

EM as a parent process

- EM as the parent process can help child process and take the appropriate platform-specific actions such as processing exe- cution dependencies that rely on the Terminated state and thus ensure that there is no overlap between these processes when both are running. -> but I think that we don’t need it because nm don’t rely on the others module

fork is a c/c++ function in system lib to make a child process

Another way of reporting

- Execution Managementdifferentiatesbetweentwotypesofprocesses:Report- ing Processes and Non-reporting Processes. Reporting Processes are considered to be the normal form of processes and Non-reporting Processes are considered to be an exception.

- Non-reporting Processes can be used to support running Executables which have not been designed with the AUTOSAR Adaptive Platform in mind. For example, if an Executable is available as binary only, if it is not feasible to patch its source code or if the Executable is only used during development time.

os run adaptive autosar main process that runs the em process that communicate with the sm that runs the process state like (running)

how this done or implemented ?
throw threading
client and server threads that communicate with each other
em manager that communicate with sm

through IPC, could be message passing, message queues , future and promise, shared memory.

how to start and stop applications using system lib, and how to look like in em

we have standard in Adaptitve autosar for machine state

this implemnted using threads